Autoencoders

Métodos Generativos, curso 2026-2027

Guillermo Iglesias, guillermo.iglesias@upm.es
Edgar Talavera Muñoz, e.talavera@upm.es

7 de noviembre de 2026

Escuela Técnica Superior de Ingeniería de Sistemas Informáticos | UPM



¿Qué es un Autoencoder? (Visión formal)

Tradicionalmente se definen como redes neuronales que aprenden a comprimir y reconstruir datos. A nivel matemático, un Autoencoder (AE) es un modelo diseñado para aprender la función identidad $f(x) \approx x$ bajo una serie de restricciones.

El modelo se divide en dos funciones parametrizadas (normalmente redes neuronales):

- **Codificador (Encoder):** Mapea los datos de entrada a un espacio latente (o *bottleneck*). Se define como $z = f_\theta(x)$.
- **Decodificador (Decoder):** Intenta reconstruir la entrada original a partir del espacio latente. Se define como $\hat{x} = g_\phi(z)$.

El objetivo es optimizar los parámetros θ y ϕ para minimizar el Error de Reconstrucción entre x y $\hat{x}$.

¿Para qué se emplean?

Son muy útiles en una amplia variedad de aplicaciones.

- Reducción de dimensionalidad de datos de alta dimensión.
- Eliminación de ruido en señales.
- Detección de anomalías.
- Generación de nuevos datos, similares a los datos de entrada[1].

[1] Ian Goodfellow menciona que son la **primera red generativa**.

Relevancia en la actualidad: ¿Por qué estudiarlos?

Aunque los AE clásicos se propusieron inicialmente para reducción de dimensionalidad o eliminación de ruido [1], hoy en día son la piedra angular del Deep Learning Generativo:

- **Modelos de Difusión Latente (LDM):** Arquitecturas como Stable Diffusion usan espacios latentes z generados por un Autoencoder preentrenado.
- **Modelos Multimodales:** Utilizan las proyecciones del *encoder* para alinear texto, imagen y audio en un mismo espacio dimensional (e.g., Contrastive Language-Image Pre-training (CLIP)[2])[2].

[2] Más adelante iremos conociendo estas arquitecturas, pero vamos, que los AE se usan en todos lados.

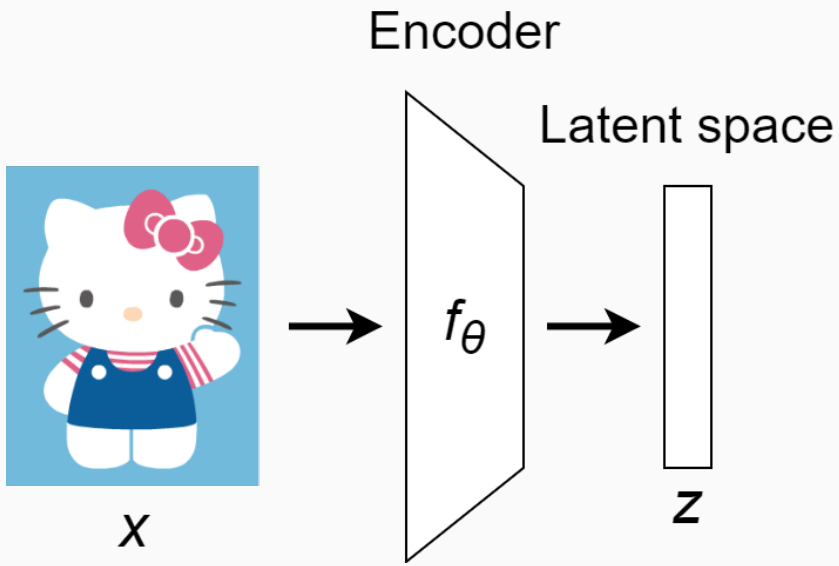
Arquitectura: Formalización Matemática

En una AE definimos un espacio de entrada $\mathcal{X} \in \mathbb{R}^D$ y un espacio latente $\mathcal{Z} \in \mathbb{R}^d$, donde $d \ll D$.

La arquitectura se define formalmente mediante dos funciones parametrizadas:

- Codificador (f_θ): Transforma la entrada x en el vector latente.

$$z = f_\theta(x)$$



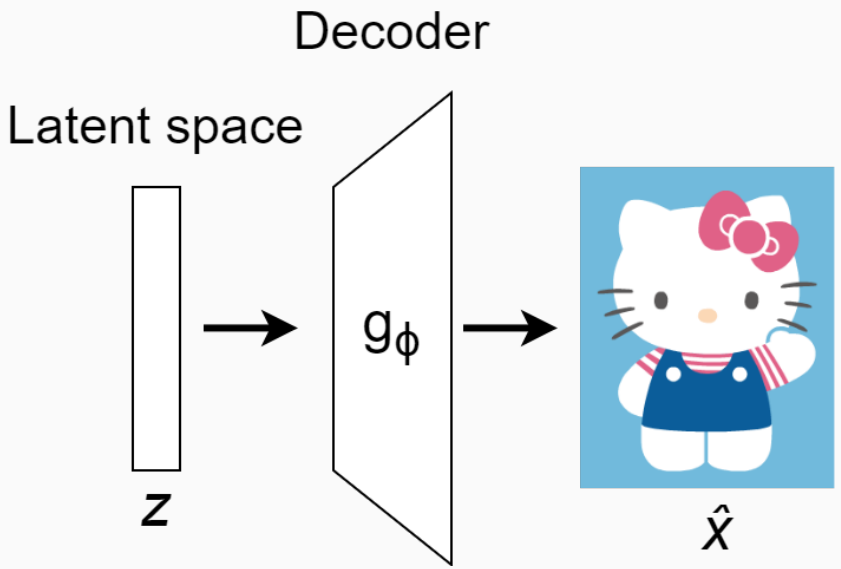
Arquitectura: Formalización Matemática

- Decodificador (g_ϕ): Reconstruye la entrada a partir del vector latente z.

$$\hat{x} = g_\phi(z)$$

El proceso completo (la predicción del modelo) es la composición de ambas funciones:

$$\hat{x} = g_\phi(f_\theta(x))$$



Entrenamiento y Función de Pérdida

Como en un problema de regresión, el objetivo del entrenamiento es encontrar los parámetros óptimos θ^* y ϕ^* que minimicen una función de pérdida $\mathcal{L}$.

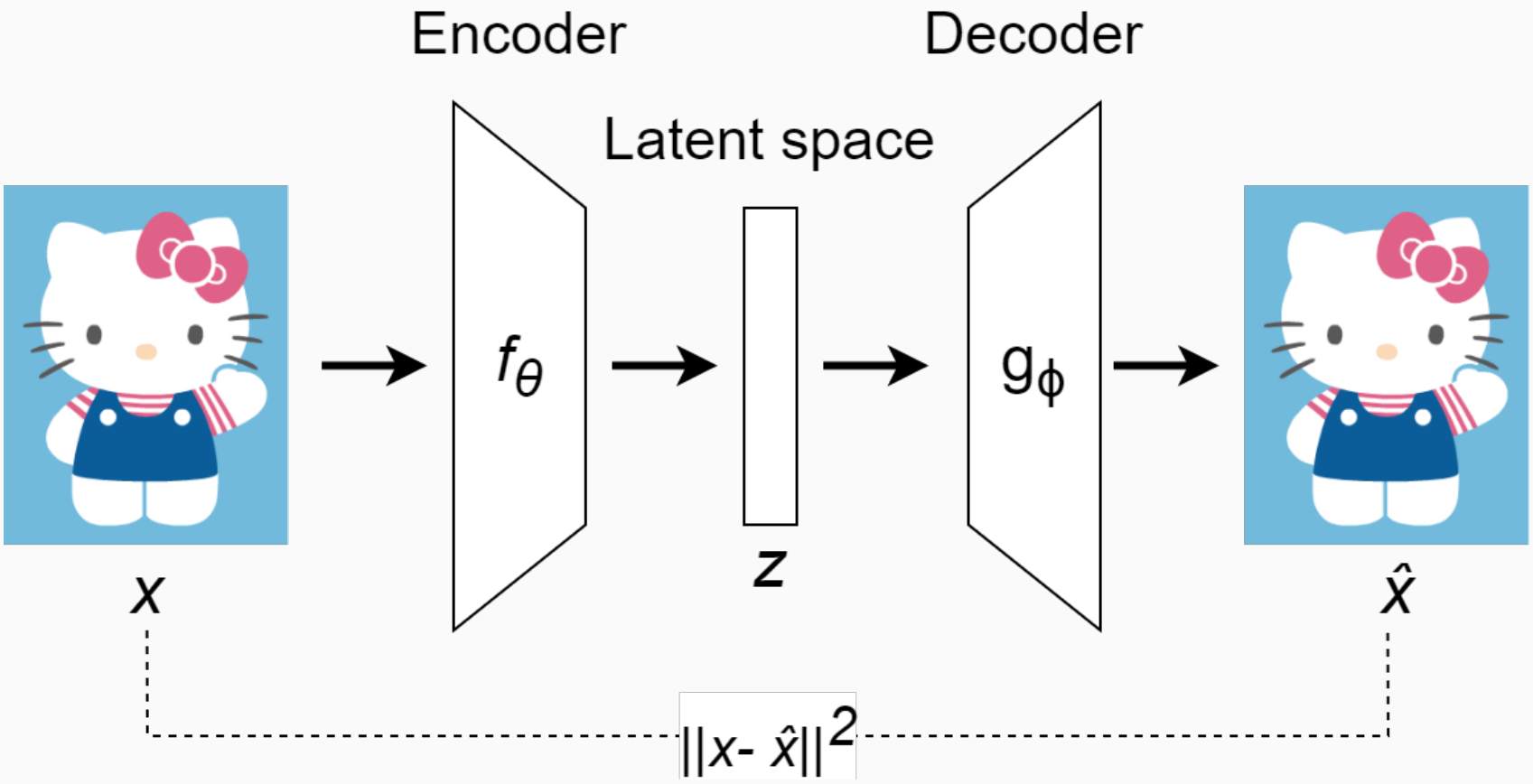
La función de pérdida mide el error de reconstrucción entre la entrada original x y la reconstrucción $\hat{x}$. Dependiendo del dato se puede usar como loss:

- **Error Cuadrático Medio (MSE):** Estándar para datos continuos.

$$\mathcal{L}(\theta, \phi) = \frac{1}{N} \sum_{i=1}^{N} ||x_i - g_\phi(f_\theta(x_i))||^2$$

- **Entropía Cruzada Binaria (BCE):** Usada cuando las entradas están normalizadas en el rango $[0, 1]$ (tratando los píxeles como probabilidades).

Arquitectura: Esquema



Capacidad del Modelo: El problema de la identidad

Dependiendo de la dimensionalidad del espacio latente podemos tener un espacio:

- **Undercomplete ($d < D$):** Al pasar por un cuello de botella, se extraen los rasgos más relevantes de los datos.
- **Overcomplete ($d \geq D$):** El codificador/decodificador tienen demasiados parámetros, la red puede colapsar aprendiendo la función identidad ($f(x) = x$) de memoria, sin aprender ninguna representación útil[3].

[3] Inútil? Puede parecer, pero en ciertas arquitecturas modernas [3] son muy populares.

Manifold Hypothesis

La clave reside en el cuello de botella, que fuerza a la red a descartar ruido y redundancias, apoyándose en la Manifold Hypothesis:

- Los datos del mundo real (como una imagen de *Hello Kitty* de 256×256 píxeles) ocupan de dimensiones altísimas, pero no están distribuidos uniformemente.
- En realidad, residen en una subvariedad (*manifold*) de dimensión mucho menor[4].
- El Autoencoder aprende la topología de esas subvariedades, comprimiendo el espacio.

[4] Es decir, representan conceptos más sencillos.

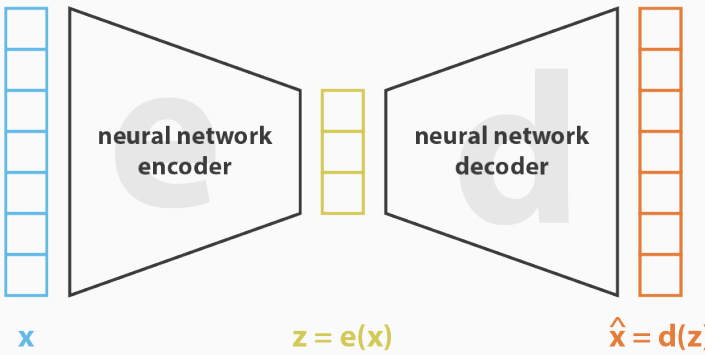
Arquitectura: Representación del espacio latente

El espacio latente será capaz de representar los datos de manera ordenada, extrayendo sus características de alto nivel. Este nivel de abstracción es el que buscamos.



[4]

¿Por qué necesitamos Autoencoders?



$$\text{loss} = \| x - \hat{x} \|^2 = \| x - d(z) \|^2 = \| x - d(e(x)) \|^2$$

Los autoencoders tienen muchas aplicaciones en el mundo real:

- Compresión y reconstrucción de imágenes.
- Generación de música y separación de fuentes de sonido.
- Compresión de archivos para reducir el tamaño sin perder información importante.
- Extracción de características numéricas de señales de voz que se pueden usar para identificar palabras y frases.

Tipos de autoencoders

Los autoencoders son modelos que tienen muchas variaciones. Las más comúnmente utilizadas son:

- Vanilla Autoencoder
- Stacked autoencoders
- Denoising Autoencoder
- Convolutional Autoencoder
- *Variational Autoencoder*[5]

[5] Los veremos más adelante, ya que introducen variaciones importantes en la arquitectura básica de un autoencoder.

Comprimiendo datos con Vanilla Autoencoder

Limitaciones de los Vanilla Autoencoder

Los Vanilla Autoencoder no son muy eficientes en la generación de datos sintéticos.

Problema principal: El **espacio latente generado no es continuo**.

- Está compuesto por regiones separadas entre sí que agrupan características de ejemplos similares.
- Entre estas regiones no hay un espacio continuo de representaciones intermedias.
  - En realidad, lo hay, pero no tiene sentido.
  - ¿Realmente no hay un espacio intermedio entre un 1 y un 7? ¿o entre un 3 y un 8?
- Esto hace que la interpolación entre ejemplos sea imposible.

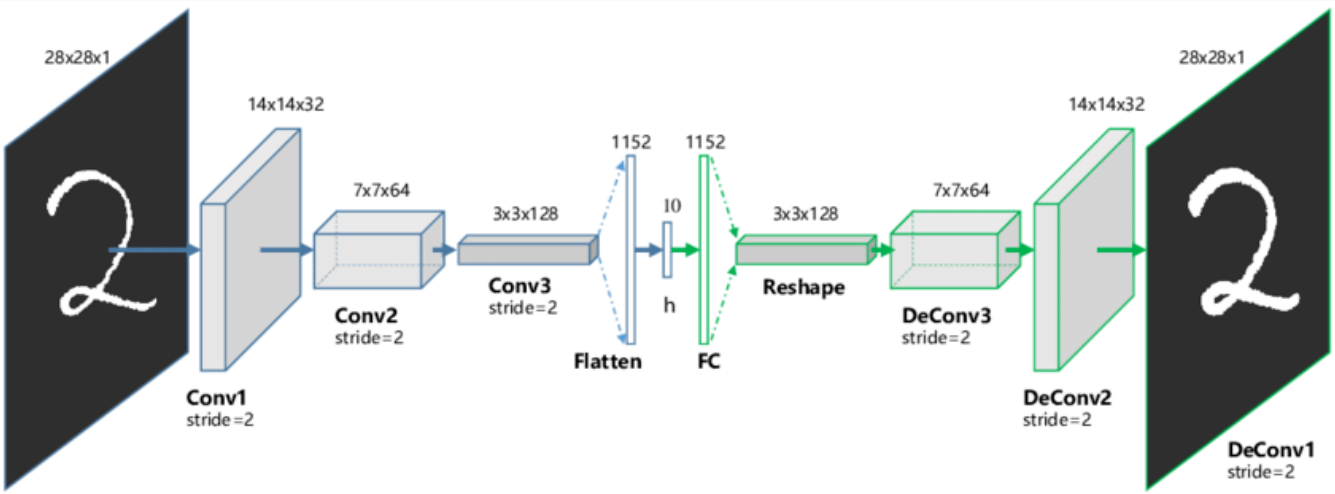
En resumen, **si el espacio intermedio no es continuo, las salidas del decodificador no son realistas**. –> Esto lo solucionan los autoencoders variacionales

Autoencoders Convolucionales

Un tipo de red neuronal diseñado para procesar y reconstruir datos de imágenes.

- Utilizan Convolutional Neural Networks (CNNs) debido a su capacidad para capturar relaciones espaciales entre píxeles.
- El codificador típicamente consta de varias capas convolucionales, seguidas de un conjunto de capas totalmente conectadas.
  - Las capas convolucionales extraen características de la imagen de entrada.
  - Las capas totalmente conectadas luego combinan estas características en una representación de menor dimensión de la imagen.
- El decodificador es esencialmente un espejo del codificador, con un conjunto de capas totalmente conectadas seguidas de varias capas deconvolucionales.

Autoencoders Convolucionales



Esquema de un Autoencoder convolucional (fuente).

Pueden aprender a representar datos de imágenes de manera más compacta y eficiente

- Reduciendo así la cantidad de memoria y potencia de procesamiento necesarias para tareas posteriores como clasificación de imágenes o detección de objetos.

Autoencoders convolucionales

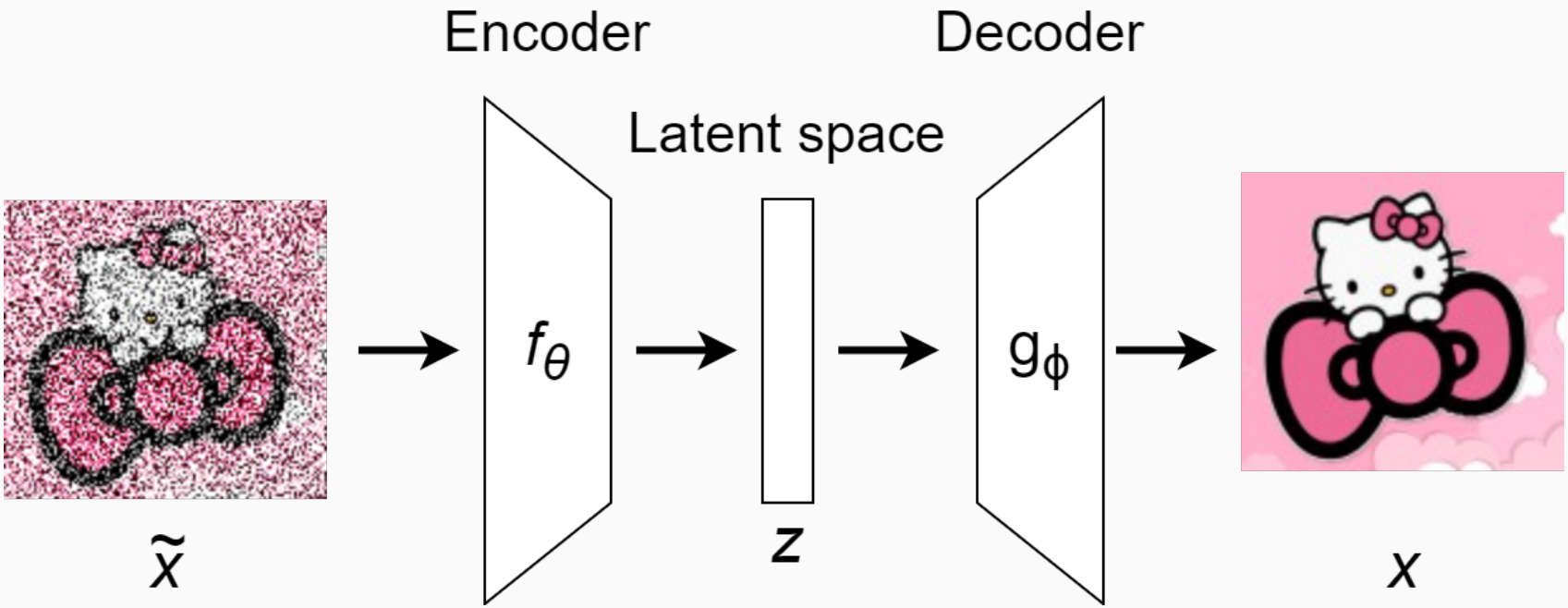
La flexibilidad del paradigma Encoder-Decoder

En la arquitectura Encoder-Decoder con cuello de botella se pueden modificar pares de **Entrada** / **Salida Esperada** y la **Función de Pérdida**, pudiendo tratar problemas radicalmente distintos, entre otros:

- **Compresión:** Entrada x, Salida x. (AE clásico).
- **Eliminación de ruido:** Entrada $\tilde{x}$ (corrupta), Salida x (limpia).
- **Segmentación semántica:** Entrada x (foto), Salida y (máscara).
- **Coloreado:** Entrada x (blanco y negro), Salida y (color).

Autoencoders para denoising

Los Autoencoders de Eliminación de Ruido [5] son capaces de recuperar una imagen original x a través de una versión corrupta $\tilde{x}$.



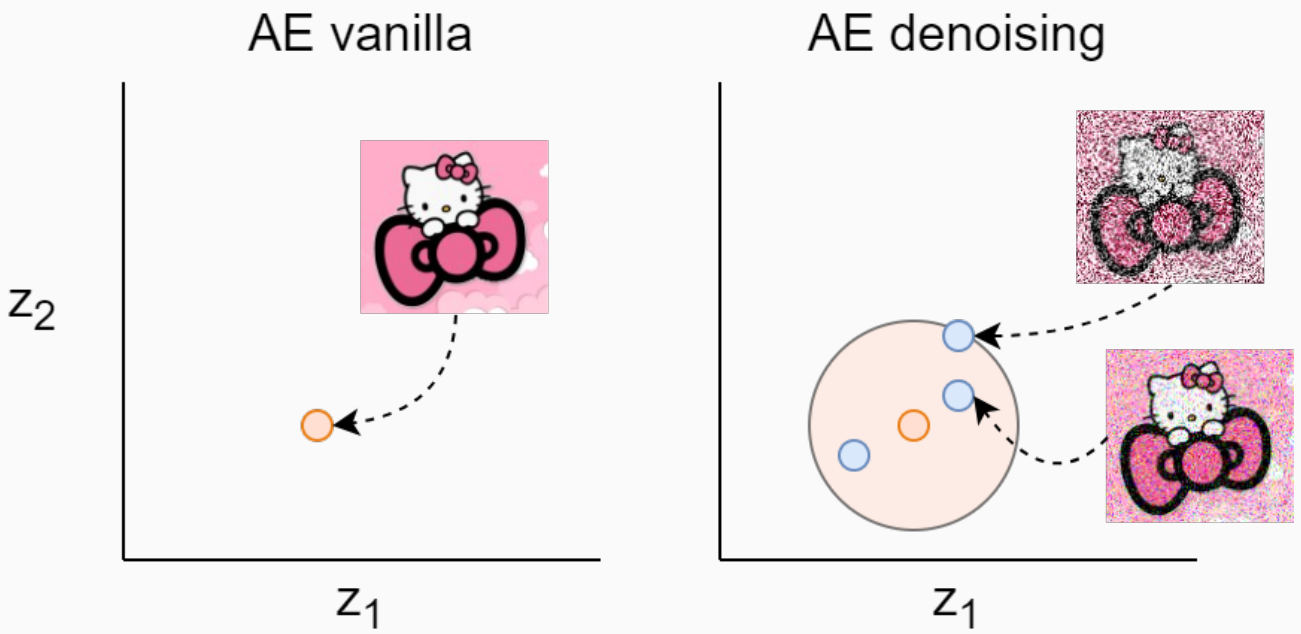
La función objetivo se modifica para que la red reconstruya la imagen limpia original (x), no la corrupta ($\tilde{x}$):

$$\mathcal{L} = ||x - g_\phi(f_\theta(\tilde{x}))||^2 \tag{1}$$

Denoising Autoencoders: Topología y Manifold

Al corromper la imagen original x, la estamos ”empujando” fuera de la subvariedad (manifold) donde residen las imágenes reales.

- El modelo aprende un campo vectorial continuo.
- Este campo vectorial apunta desde cualquier punto ruidoso hacia el punto más cercano de los datos reales.
- Genera representaciones latentes más robustas y menos sensibles a pequeñas variaciones.

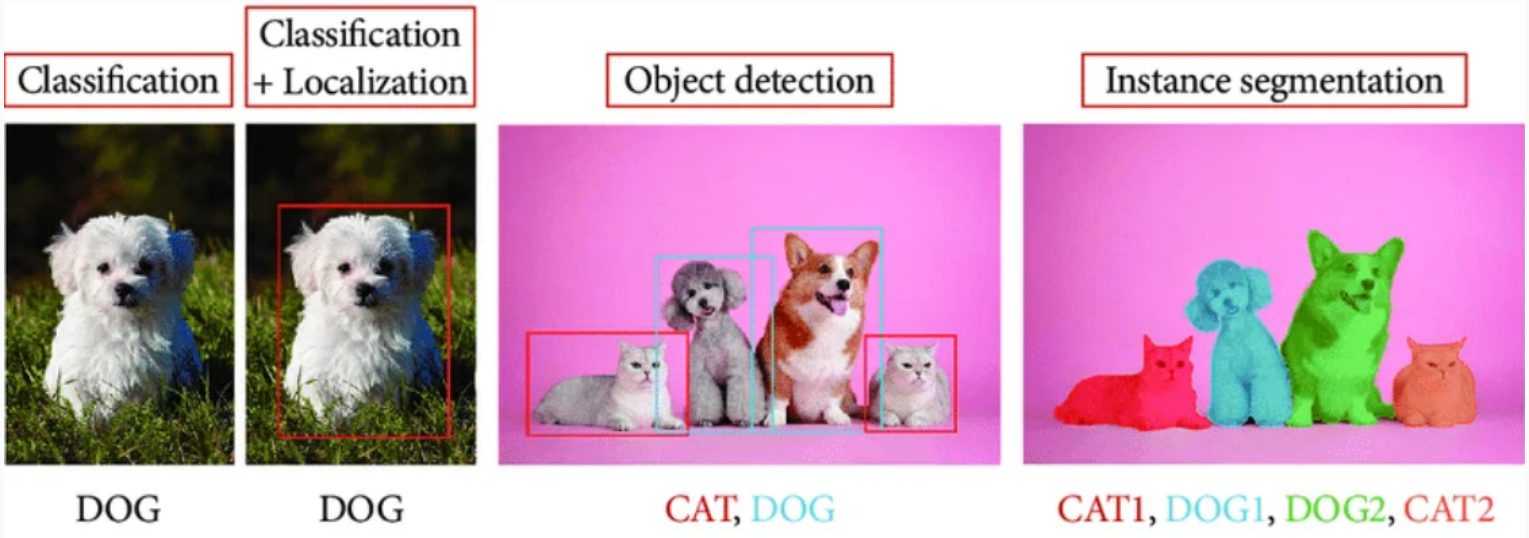


Supresión de ruido en imágenes con autoencoders de eliminación de ruido

Segmentación de imagen

El campo de la segmentación de imagen es muy relevante en visión por computador.

- La dimensionalidad de entrada es igual (o similar) a la de salida.
- Se puede ver cómo un problema de clasificación a nivel de píxel, o de traducción.



[6]

Autoencoders para segmentación semántica

Los Autoencoders son la red más habitual dentro de problemas de segmentación.

- **Entrada (x):** Una imagen RGB ($H \times W \times 3$).
- **Salida esperada (y):** Una máscara de segmentación ($H \times W \times C$), donde C es el número de clases, indicando a qué objeto pertenece cada píxel.
- **Función de Pérdida:** Ya no usamos el Error Cuadrático (MSE), sino Entropía Cruzada (Cross-Entropy) a nivel de píxel.

El cuello de botella fuerza a la red a extraer el contexto global (qué objetos hay), y el decodificador utiliza esa información semántica para proyectar la máscara precisa a la resolución original.

El problema de la resolución espacial

En un Autoencoder clásico, la imagen pasa por un cuello de botella muy severo:

- El codificador extrae un excelente contexto semántico (*¿Qué objetos hay en la imagen?*).
- Pero al reducir drásticamente la dimensionalidad, se destruye la información espacial fina (*¿Dónde están exactamente los bordes de esos objetos?*).

Para solucionar este conflicto en 2015 se propuso la U-Net [7][6].

[6] Originalmente se proponen para segmentación de imagen biomédica, pero la vida da muchas vueltas.

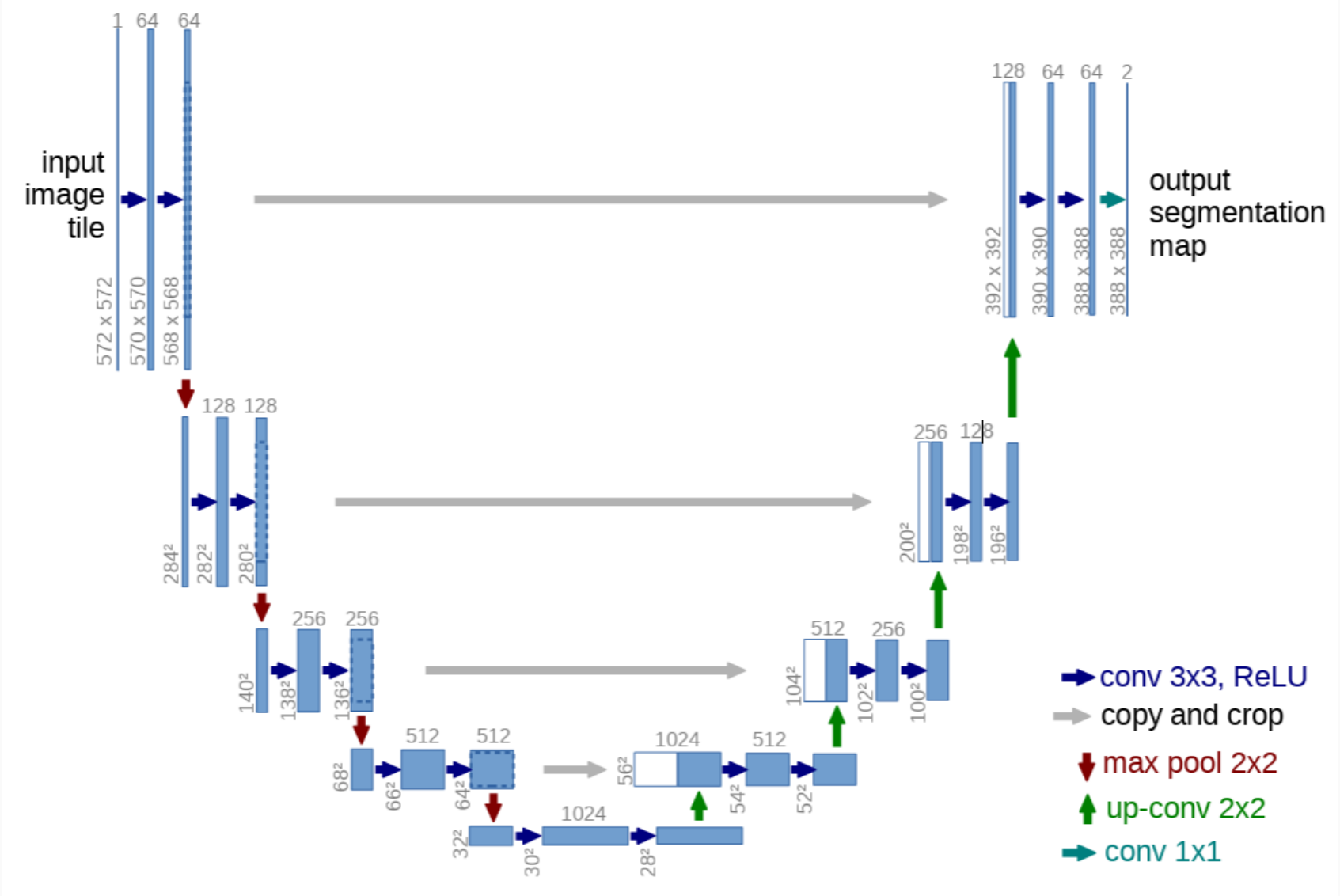
Arquitectura U-Net

La U-Net introduce las Skip Connections que unen capas del mismo nivel de resolución.

Se divide en dos rutas simétricas:

- **Ruta de contracción (Encoder):** se reducen las dimensiones espaciales y aumentan los canales. Extrae características semánticas de alto nivel.
- **Ruta de expansión (Decoder):** Se que recupera progresivamente la resolución espacial.

Arquitectura U-Net



[7]

Skip Connections: La clave matemática

La innovación de la U-Net es cómo une el Encoder y el Decoder. En cada nivel, el mapa de características del Encoder se concatena (no se suma) con el mapa del Decoder.

A nivel de tensores, si el Decoder genera un tensor de tamaño $H \times W \times C_1$ y recibe por la conexión residual un tensor del Encoder de tamaño $H \times W \times C_2$, la concatenación en la dimensión de los canales produce:

$$T_{out} = H \times W \times (C_1 + C_2) \tag{2}$$

Esto permite que la red convolucional posterior combine:

- La información semántica profunda del Decoder.
- La información espacial de alta resolución intacta del Encoder.

La U-Net como motor Generativo

Aunque la U-Net nació para segmentación de imágenes médicas, su capacidad para procesar imágenes preservando detalles a nivel de píxel la ha convertido en la arquitectura más importante del Deep Learning Generativo.

Como se estudiará (o se ha estudiado) en otros temas, la U-Net es el motor principal de:

- **pix2pix (Conditional GANs):** Genera imágenes fotorrealistas a partir de bocetos, manteniendo la estructura geométrica original gracias a las *skip connections*.
- **Modelos de Difusión (DDPM, Stable Diffusion):** Se utiliza para predecir el tensor de ruido $\epsilon_\theta(x_t, t)$, que tiene exactamente la misma dimensionalidad espacial que la imagen original.

Recursos

- Diapositivas de Moodle
- Google Collaboratory
- Deep Learning Book (https://www.deeplearningbook.org/)
- https://www.pyimagesearch.com/blog
- https://machinelearningmastery.com/blog

Referencias i

[1] Dor Bank, Noam Koenigstein, and Raja Giryes.
Autoencoders. arxiv.
arXiv preprint arXiv:2003.05991, pages 2593–2613, 2020.

[2] Alec Radford, Luke Metz, and Soumith Chintala.
Unsupervised representation learning with deep convolutional generative adversarial networks.
arXiv preprint arXiv:1511.06434, 2015.

[3] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie.
A convnet for the 2020s.
In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022.

Referencias ii

[4] Julien Despois (HACKERNOON).
Mnist latent space image.
https://hackernoon.com/latent-space-visualization-deep-learning-bits-2-bd09a46920df, February 23rd, 2017.
[Online; accessed July, 2026].

[5] Pascal Vincent, Hugo Larochelle, Yoshua Bengio, and Pierre-Antoine Manzagol.
Extracting and composing robust features with denoising autoencoders.
In *Proceedings of the 25th international conference on Machine learning*, pages 1096–1103, 2008.

Referencias iii

[6] Joonmyeong Choi, Keewon Shin, Jinhoon Jung, Hyun-Jin Bae, Do Hoon Kim, Jeong-Sik Byeon, and Namku Kim.
Convolutional neural network technology in endoscopic imaging: artificial intelligence for endoscopy.
Clinical endoscopy, 53(2):117–126, 2020.

[7] Olaf Ronneberger, Philipp Fischer, and Thomas Brox.
U-net: Convolutional networks for biomedical image segmentation.
In *International Conference on Medical image computing and computer-assisted intervention*, pages 234–241. Springer, 2015.

Citación

Cita APA: Iglesias, G. and Talavera, E. (2026, noviembre 7). *Autoencoders* [Diapositivas de la asignatura Métodos Generativos]. GitHub. *https://guillermoih.github.io/M-todos-Generativos/Autoencoders.pdf*

Tema de las diapositivas "Metropolis" lincenciado bajo LPPL 1.3c

BibTeX

```
@slides{Autoencoders_MG_2026,
   author = {Iglesias, Guillermo and Talavera, Edgar},
   title = {Autoencoders [Diapositivas de la asignatura Métodos Generativos]},
   year = {2026},
   month = {11},
   howpublished={\url{https://guillermoih.github.io/M-todos-Generativos/Autoencoders.pdf}},
   note = {[Online; accessed 7 de noviembre de 2026]},
   organization={ETSISI. Universidad Politécnica de Madrid}
}
```